

Versão 2026 — Estrutura Validada

Guia Completo de Replicação

Especificação integral e atualizada da loja TopMix: interface, checkout, persistência server-side, pagamentos, conciliação, atribuição UTM, admin, rastreo e recuperação.

React + Vite + TypeScript

Cloudflare Pages

Supabase

IronPay · MasterFy · UmbrellaPag · Venus Pay

Atualizado em 04/09/2026 · Revisado contra o código-fonte atual. Para uma cópia realmente idêntica, forneça este guia e os ativos autorizados da nova loja.

Índice

1. Visão geral da arquitetura
2. Estrutura de pastas do projeto
3. Passo a passo — criar a loja do zero
4. Supabase — tabelas, SQL e backup
5. Cloudflare Pages — deploy, env vars e KV
6. Gateways de pagamento (PIX e cartão)
7. Fluxos principais (compra, rastreio, admin)
8. O que personalizar na nova loja
9. Lógica que deve ser implementada exatamente
10. Serviços externos e contas necessárias
11. Checklist de testes antes de ir ao ar
12. Prompt pronto para usar com um assistente de IA
13. Problemas comuns e soluções
14. Contratos de dados, estados e regras de idempotência
15. Controle de versão e critério de conclusão

1. Visão geral da arquitetura

Resumo: Loja e-commerce completa com checkout próprio, criação server-side de pedidos, painel admin, conciliação automática de PIX, rastreo simulado, emails automáticos, recuperação de carrinho e integração com Facebook CAPI, dataLayer/GA e UTMify. Produtos ficam em arquivo estático (`src/lib/data.ts`); pedidos e atribuição ficam no Supabase.

O que mudou nesta revisão:

Pedido passa a ser criado por `/api/orders/create` ; atribuição preserva UTMs por 30 dias; PIX é conciliado no servidor; webhook nunca confia apenas no corpo recebido; admin atualiza silenciosamente a cada 60 segundos; UTMify recebe `waiting_payment` e depois `paid` ; barra de frete usa GPS preciso com fallback controlado por IP.

server-side leads

UTM first/last touch

reconciliação PIX

admin sem piscar

geolocalização ≤ 500 m

Camada	Tecnologia	Função
Frontend	React 19 + Vite 7 + TypeScript + Tailwind 4 + shadcn/ui	Site da loja, checkout, admin, rastreo
Backend	Cloudflare Pages Functions (<code>functions/api/</code>)	Pedidos, PIX, cartão, conciliação, webhooks, emails, cron, FB CAPI e UTMify
Banco	Supabase (PostgreSQL)	leads, payment_gateways, rastreo_origem, comprovantes
Hospedagem	Cloudflare Pages	Build <code>dist/</code> + functions na edge
Rate limit PIX	Cloudflare KV (<code>PIX_RATELIMIT</code>)	Máx. 5 PIX por IP/hora
Emails	Resend	Rastreo + recuperação de carrinho
Cron	cron-job.org (gratuito)	Fila de recuperação de carrinho a cada 15 min

Diagrama do fluxo

```
[Cliente] → Site React (Cloudflare Pages)
      ↓ valida formulário + captura tracking
      /api/orders/create → Supabase (leads)
      ↓ orderId obrigatório
      /api/pix/create ou /api/card/create
      ↓
      IronPay / MasterFy / UmbrellaPag / Venus Pay
      ↓ polling / webhook validado / reconciliação do admin
      Gera código rastreio → rastreio_origem
      ↓
      Email Resend + FB Purchase + UTMify (deduplicado)
      ↓
      Cliente rastreia em /rastrear-pedido
```

Rotas do site (frontend)

URL	Arquivo	Descrição
/	home.tsx	Página inicial, hero, produtos, vídeo
/produto/:slug	product.tsx	Página do produto com galeria e avaliações
/carrinho	cart.tsx	Carrinho (localStorage)
/checkout	checkout.tsx	Checkout PIX/cartão — cria lead
/sucesso	success.tsx	Pós-compra (aprovado/pendente/recusado)
/rastrear-pedido	rastrear-pedido.tsx	Timeline de rastreio simulada
/admin	admin.tsx	Painel administrativo completo
/duvidas-frequentes, /fale-conosco, /politica-de-trocas	institucionais	Páginas de suporte

APIs do backend (Cloudflare Functions)

Rota	Método	Função
/api/admin-login	POST	Login admin → token HMAC (8h)
/api/admin-verify	GET	Valida token do admin
/api/admin-gateways	GET/PATCH	Listar/ativar gateways (admin)
/api/orders/create	POST	Cria o pedido no servidor e resolve a atribuição UTM
/api/pix/create	POST	Gera cobrança PIX (gateway ativo)
/api/pix/status	GET	Consulta status do PIX
/api/pix/webhook	POST	Webhook dos gateways PIX
/api/card/create	POST	Pagamento cartão (Venus Pay)
/api/card/status	GET	Status do cartão
/api/send-tracking-email	POST	Email com código de rastreio
/api/send-recovery-email	POST	Email de carrinho abandonado
/api/process-recovery-queue	POST	Cron de recuperação (cron-job.org)
/api/fb-purchase	POST	Facebook Conversions API
/api/utmify-order	POST	Sincroniza pedido na UTMify
/api/reconcile-payments	POST	Confere até 50 PIX pendentes dos últimos 7 dias; aceita token admin ou segredo do cron

2. Estrutura de pastas do projeto

```
loja/
├─ index.html           ← Título, meta tags, GTM, GA4, UTMify pixel
├─ package.json/package-lock ← Dependências fixadas + scripts dev/build/serve/typecheck
├─ vite.config.ts       ← Vite + Tailwind + alias @/
├─ wrangler.toml        ← Cloudflare Pages + KV + lista de env vars
├─ public/
│   └─ images/          ← Fotos dos produtos, logo, hero, avaliações e vídeo
├─ src/
│   └─ App.tsx           ← Rotas (wouter)
│   └─ lib/
│       └─ data.ts       ← ★ PRODUTOS (nome, preço, fotos, reviews)
│       └─ supabase.ts   ← Cliente Supabase + tipos Lead
│       └─ encrypt.ts    ← Criptografia AES dos dados de cartão
│       └─ tracking.ts    ← UTM, GA client_id, eventos ecommerce
│       └─ utils.ts      ← Utilitários gerais
│   └─ hooks/use-cart.tsx ← Carrinho global (localStorage)
│   └─ components/       ← UI, header, footer, admin, vídeo lazy e barra de frete
│       └─ pages/        ← Todas as páginas listadas acima
├─ functions/
│   └─ api/              ← Rotas HTTP (Cloudflare Pages Functions)
│       └─ _lib/
│           └─ admin-auth.js ← Autenticação do admin
│           └─ gateway-config.js ← Lê/toggle gateways no Supabase
│           └─ leads-store.js ← Persistência server-side, atribuição e fulfillment PIX
│           └─ pix-gateway-status.js ← Normalização de status dos 4 gateways PIX
│           └─ utmify.js     ← Payload único e validado da UTMify
│           └─ pix-gateways/ ← Adaptadores IronPay, MasterFy, UmbrellaPag, Venus
└─ supabase/
    └─ payment_gateways.sql ← Tabela + seed dos 5 gateways
    └─ leads_add_gateway.sql ← Coluna gateway na tabela leads
    └─ payment_gateways_add_umbrellapag.sql
    └─ payment_gateways_add_venuspay_pix.sql
```

3. Passo a passo — criar a loja do zero

Exemplo: Loja de capacetes. A IA (ou o desenvolvedor) cria um projeto NOVO seguindo este guia, com os produtos, fotos, textos, credenciais e domínio do dono da loja. Não copiar visual, produtos nem identidade de nenhum outro site.

1

Criar o projeto a partir deste guia

Este PDF é a especificação completa. Crie o código do zero (estrutura da Seção 2). Não use zip, clone nem código de outra loja.

```
npm install
npm run dev
```

Abra `http://localhost:5173`. O site roda localmente, mas pagamentos e admin precisam das variáveis de ambiente.

2

Criar projeto Supabase (banco novo)

1. Acesse **supabase.com** → New Project
2. Anote: **Project URL**, **anon key**, **service_role key**
3. SQL Editor → execute os scripts da Seção 4 (tabelas + payment_gateways)
4. Storage → crie bucket público `comprovantes` (para upload de comprovantes na página de rastreio)
5. Confirme que as colunas de endereço, tracking, gateway, recuperação e idempotência existem exatamente como na Seção 4

3

Personalizar produtos e identidade visual

Veja Seção 8 completa. Resumo: editar `src/lib/data.ts`, imagens em `public/images/`, textos em `header/footer/home/index.html`.

4

Criar projeto Cloudflare Pages

1. Acesse **pages.cloudflare.com** → Create a project → Connect to Git
2. Conecte o repositório **novo** da loja (conta Git do dono desta loja)
3. Build command: `npm run build`
4. Build output: `dist`
5. Node.js version: **20**

5

Configurar variáveis de ambiente no Cloudflare

Settings → Environment variables → Production. Adicione TODAS as variáveis da Seção 5. Variáveis `VITE_*` são injetadas no build.

6

Criar KV namespace para rate limit PIX

1. Cloudflare → Workers & Pages → KV → Create namespace → nome: `pix-ratelimit`
2. Pages → Settings → Functions → KV namespace bindings
3. Variable name: `PIX_RATELIMIT` → namespace: `pix-ratelimit`

7

Configurar gateways de pagamento

Crie contas nos provedores, obtenha credenciais e configure webhooks. Detalhes na Seção 6.

8

Configurar Resend (emails)

1. Crie conta em **resend.com**
2. Verifique seu domínio (DNS: SPF, DKIM)
3. API Key → `RESEND_API_KEY`
4. Email remetente → `RESEND_FROM_EMAIL` (ex: contato@sualoja.com.br)

9

Configurar cron de recuperação de carrinho

1. Crie conta em **cron-job.org** (gratuito)
2. Novo job: POST `https://SEU-DOMINIO/api/process-recovery-queue`
3. Header: `x-cron-secret: SEU_CRON_SECRET`
4. Intervalo: a cada 15 minutos

10**Deploy e domínio**

1. Push para `main` → Cloudflare faz deploy automático
2. Custom domains → adicione seu domínio (ex: loja-capacetes.com.br)
3. Atualize `SITE_URL` com a URL final
4. Atualize webhooks dos gateways com a URL final

11**Testar tudo (Seção 11)**

PIX, cartão, criação server-side do pedido, conciliação, admin, rastreo, emails, atribuição UTM, gateways e localização — antes de anunciar.

4. Supabase — tabelas, SQL e backup

4.1 Tabela `leads` (todos os pedidos)

Execute no SQL Editor do Supabase (projeto novo, vazio):

```
CREATE TABLE IF NOT EXISTS leads (  
  id                BIGSERIAL PRIMARY KEY,  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  nome              TEXT NOT NULL,  
  email             TEXT NOT NULL,  
  telefone          TEXT NOT NULL,  
  cpf               TEXT,  
  produtos          TEXT NOT NULL,  
  valor             NUMERIC(10,2) NOT NULL,  
  metodo_pagamento TEXT NOT NULL,  
  status            TEXT NOT NULL DEFAULT 'checkout_iniciado',  
  transaction_id    TEXT,  
  gateway           TEXT,  
  codigo_rastreio   TEXT,  
  card_encryptado   TEXT,  
  card_erro         TEXT,  
  ga_client_id      TEXT,  
  purchase_sent     BOOLEAN DEFAULT FALSE,  
  tracking           JSONB,  
  cidade            TEXT,  
  estado            TEXT,  
  cep               TEXT,  
  rua               TEXT,  
  numero            TEXT,  
  complemento       TEXT,  
  bairro            TEXT,  
  recovery_count    INTEGER DEFAULT 0,  
  recovery_next_at  TIMESTAMPTZ  
);  
  
ALTER TABLE leads ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "leads_all" ON leads FOR ALL USING (true) WITH CHECK (true);
```

Status usados: `checkout_iniciado`, `pix_gerado`, `pago`, `expirado`, `cartao_processando`, `cartao_recusado`, `reembolsado`. O rótulo antigo `abandonou` continua apenas para compatibilidade com registros legados e não deve aparecer como nova opção.

Contrato do campo `tracking`

```
{
  "utm_source": string | null,
  "utm_medium": string | null,
  "utm_campaign": string | null,
  "utm_content": string | null,
  "utm_term": string | null,
  "gclid": string | null,
  "fbclid": string | null,
  "ttclid": string | null,
  "first_touch": { ...os 8 campos acima },
  "last_touch": { ...os 8 campos acima } | null,
  "attribution_source": "current_visit" | "recovered_customer_history" | "unattributed",
  "recovered_via_whatsapp": boolean,
  "attribution_window_days": 30
}
```

Na criação do pedido, o servidor procura por email ou telefone nos últimos 30 dias. Se a visita atual não tiver parâmetros, herda a atribuição mais recente do cliente; se houver parâmetros atuais, eles prevalecem e o histórico é mantido em `first_touch`.

4.2 Tabela `rastreio_origem` (obrigatória para rastreio)

```
CREATE TABLE IF NOT EXISTS rastreio_origem (
  codigo          TEXT PRIMARY KEY,
  origem_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  nome_cliente    TEXT
);

ALTER TABLE rastreio_origem ENABLE ROW LEVEL SECURITY;
CREATE POLICY "rastreio_all" ON rastreio_origem FOR ALL USING (true) WITH CHECK (true);
```

⚠ Regra crítica: A página `/rastrear-pedido` só funciona se o código existir em `rastreio_origem`. Ao marcar pedido como pago (admin ou checkout automático), salve o código em **duas** tabelas: `leads.codigo_rastreio` + `rastreio_origem`.

4.3 Tabela `payment_gateways`

```
CREATE TABLE IF NOT EXISTS payment_gateways (  
  id          TEXT PRIMARY KEY,  
  name        TEXT NOT NULL,  
  method      TEXT NOT NULL CHECK (method IN ('pix', 'card')),  
  enabled     BOOLEAN NOT NULL DEFAULT false,  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
INSERT INTO payment_gateways (id, name, method, enabled) VALUES  
  ('ironpay',    'IronPay',    'pix', true),  
  ('masterfy',   'MasterFy',   'pix', false),  
  ('umbrellapag', 'UmbrellaPag', 'pix', false),  
  ('venuspay',   'Venus Pay',   'card', true),  
  ('venuspay_pix', 'Venus Pay',   'pix', false)  
ON CONFLICT (id) DO NOTHING;  
  
ALTER TABLE payment_gateways ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "payment_gateways_all" ON payment_gateways  
  FOR ALL USING (true) WITH CHECK (true);  
  
ALTER TABLE leads ADD COLUMN IF NOT EXISTS gateway text;
```

4.4 Tabela `comprovantes_taxa` + Storage

```
CREATE TABLE IF NOT EXISTS comprovantes_taxa (  
  id          BIGSERIAL PRIMARY KEY,  
  tracking_code TEXT NOT NULL,  
  file_url    TEXT NOT NULL,  
  file_name   TEXT,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);  
  
ALTER TABLE comprovantes_taxa ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "comprovantes_all" ON comprovantes_taxa FOR ALL USING (true) WITH CHECK (true);
```

Storage → New bucket → nome `comprovantes` → Public bucket → Policies: permitir upload/read com anon key.

4.5 Regras críticas do Supabase

1. **Nunca use** `.order()` / `.limit()` junto com `.update()` — falha silenciosa. Sempre: `SELECT id → UPDATE` por id.
2. **Código de rastreio** deve ir para `leads` E `rastreio_origem`.
3. No admin, em funções async, **busque o lead direto do Supabase** — não use estado React desatualizado.
4. A criação inicial do lead é feita pelo backend com `SUPABASE_SERVICE_ROLE_KEY`; o checkout só continua se receber um `orderId`.
5. `purchase_sent` funciona como trava de idempotência da conversão UTMify após pagamento PIX. Se o envio falhar, voltar para `false` para permitir nova tentativa.
6. O fulfillment PIX só atribui rastreio quando `codigo_rastreio IS NULL`, evitando email/código duplicado em polling, webhook e reconciliação concorrentes.

Segurança obrigatória: a `SUPABASE_SERVICE_ROLE_KEY` pertence somente às Functions. Nunca usar prefixo `VITE_` nela. Para produção, substitua políticas públicas amplas por políticas RLS mínimas e endpoints autenticados. Não persista CVV após autorização e prefira tokenização do gateway para dados de cartão; a presença de criptografia no navegador não substitui PCI DSS.

4.6 Backup e restauração

Backup (Windows PowerShell):

```
$env:PGSSLMODE = 'require'
$env:PGPASSWORD = 'SENHA_DO_SUPABASE'
$h = 'db.XXXX.supabase.co'
pg_dump -h $h -U postgres -d postgres --schema=public -f ".\backup_limpo.sql"
Remove-Item Env:PGPASSWORD
```

Restaurar em projeto novo (vazio):

```
$env:PGSSLMODE = 'require'
$env:PGPASSWORD = 'SENHA_DO_PROJETO_NOVO'
$h = 'db.NOVO.supabase.co'
$arq = ".\backup_limpo.sql"
psql -h $h -U postgres -d postgres -f $arq *> ".\log_restauracao.txt"
Remove-Item Env:PGPASSWORD
```

Para loja nova sem dados antigos: use `--schema-only` no `pg_dump` + execute `payment_gateways.sql` manualmente.

Não vai no backup: arquivos do bucket Storage, variáveis Cloudflare, rascunhos do SQL Editor.

5. Cloudflare Pages — deploy, env vars e KV

5.1 Configuração de build

Campo	Valor
Framework	Vite
Build command	<code>npm run build</code>
Output directory	<code>dist</code>
Node.js	20
Functions directory	<code>functions/</code> (automático)

5.2 Todas as variáveis de ambiente

Variável	Onde	Para quê
VITE_SUPABASE_URL	Production	URL Supabase (frontend)
VITE_SUPABASE_KEY	Production	Anon key Supabase (frontend)
VITE_ENCRYPT_KEY	Production	Criptografia dos dados de cartão no admin
SUPABASE_URL	Production	URL Supabase (backend functions)
SUPABASE_SERVICE_ROLE_KEY	Production	Service role (backend — nunca expor no frontend)
ADMIN_USER	Production	Usuário do painel /admin
ADMIN_PASS	Production	Senha do painel /admin
ADMIN_SESSION_SECRET	Production	Segredo HMAC do token (64+ chars aleatórios)
IRONPAY_API_TOKEN	Production	Token IronPay (PIX)
IRONPAY_OFFER_HASH	Production	Hash da oferta IronPay
IRONPAY_PRODUCT_HASH	Production	Hash do produto IronPay
MASTERFY_API_KEY	Production	Chave Bearer MasterFy (PIX alternativo)
UMBRELLAPAG_API_KEY	Production	Chave x-api-key UmbrellaPag (PIX alternativo)
VENUS_PAY_SECRET_KEY	Production	Chave Venus Pay (cartão E PIX)
VENUS_PAY_PRODUCT_ID	Production	ID produto Venus Pay (opcional p/ cartão)
RESEND_API_KEY	Production	API key Resend
RESEND_FROM_EMAIL	Production	Email remetente (domínio verificado)
FB_PIXEL_ID	Production	ID Pixel Facebook
FB_ACCESS_TOKEN	Production	Token CAPI Facebook
UTMIFY_API_TOKEN	Production	Token UTMify
SITE_URL	Production	URL pública (webhooks, emails, cron)
CRON_SECRET	Production	Protege /api/process-recovery-queue e também autoriza /api/reconcile-payments

Após mudar qualquer variável `VITE_*` : faça um novo deploy, pois ela é incorporada ao JavaScript durante o build. Segredos sem `VITE_` ficam apenas no backend.

5.3 KV — Rate limit PIX

- Namespace: `pix-ratelimit`
- Binding: `PIX_RATELIMIT`
- Limite: 5 tentativas de PIX por IP por hora
- Chave: `ratelimit:pix:{IP}` — TTL 3600s
- Não precisa exportar dados do KV (contadores temporários)

5.4 Deploy manual (alternativa ao Git)

```
npm run build
npx wrangler pages deploy dist
```

6. Gateways de pagamento (PIX e cartão)

ID	Nome	Método	Variáveis	Webhook
ironpay	IronPay	PIX	IRONPAY_API_TOKEN, OFFER_HASH, PRODUCT_HASH	{SITE_URL}/api/pix/webhook
masterfy	MasterFy	PIX	MASTERFY_API_KEY	{SITE_URL}/api/pix/webhook
umbrellapag	UmbrellaPag	PIX	UMBRELLAPAG_API_KEY	{SITE_URL}/api/pix/webhook
venuspay	Venus Pay	Cartão	VENUS_PAY_SECRET_KEY, VENUS_PAY_PRODUCT_ID	—
venuspay_pix	Venus Pay	PIX	VENUS_PAY_SECRET_KEY	Registrar manual no painel Venus Pay

Regra: Apenas **um gateway PIX** pode estar ativo por vez. O admin (botão Gateways) desativa os outros automaticamente ao ativar um. Cartão Venus Pay é independente do PIX.

Como ativar/desativar gateways

1. Acesse `/admin` → botão **Gateways**
2. Use os toggles para ligar/desligar cada gateway
3. Gateway sem credencial no Cloudflare aparece como "não configurado"
4. Se cartão estiver desligado, checkout mostra mensagem de indisponibilidade

Configurar webhooks nos provedores

Após deploy com domínio final, configure em cada painel:

```
https://SEU-DOMINIO.com.br/api/pix/webhook
```

Venus Pay PIX: registrar manualmente no dashboard Venus Pay → API → Webhooks.

Validação do webhook: o corpo recebido serve apenas para identificar transação e provedor. Antes de marcar como pago, o backend consulta o gateway com a credencial privada. PIX pago pode ser concluído por três caminhos idempotentes: polling da tela de sucesso, webhook ou `/api/reconcile-payments`.

7. Fluxos principais

7.1 Compra via PIX

Cliente preenche checkout

- salva comprador/endereço no localStorage
- captura UTM + gclid/fbclid/ttclid + GA client_id
- POST /api/orders/create
- servidor cria lead (status: checkout_iniciado) e retorna orderId + tracking efetivo
- POST /api/pix/create com orderId (gateway ativo)
- rate limit KV: máximo 5 tentativas/IP/hora
- vincula transaction_id + gateway ao lead (status: pix_gerado)
- envia pedido waiting_payment à UTMify sem bloquear o PIX se a UTMify falhar
- Exibe QR Code + copia e cola
- Cliente paga
- Tela consulta /api/pix/status periodicamente
- Polling OU webhook validado OU reconciliação detecta pagamento
- Gera código rastreio TM + 8 caracteres sem ambiguidade
- Salva leads.codigo_rastreio + rastreio_origem
- Email automático de rastreio (Resend)
- Marca status pago e envia UTMify paid uma única vez (purchase_sent)
- Evento Purchase do frontend/CAPI conforme o fluxo configurado

Um registro por tentativa de pagamento: /api/orders/create é o único ponto de criação. No checkout normal, crie um único pedido e deixe /api/pix/create atualizar o orderId recebido. A exceção intencional é a troca após cartão recusado ou indisponível: o botão “Pagar via Pix agora” cria primeiro um **segundo pedido independente** e só então gera o PIX nesse novo orderId. O cartão recusado nunca deve ser convertido ou sobrescrito.

7.2 Compra via Cartão (Venus Pay)

Cliente preenche checkout + dados do cartão

- valida Luhn, titular, validade, CPF e CVV
- calcula parcelamento (juros a partir de 6x)
- POST /api/orders/create (status: checkout_iniciado)
- POST /api/card/create
- Aprovação instantânea → status: pago
- Gera rastreio automático
- Email + FB + UTMify
- Redireciona /sucesso?payment=card&status=approved

Se gateway responder pending:

- status cartao_processando
- /sucesso consulta /api/card/status a cada 5s

Se recusar ou ficar indisponível:

- status cartao_recusado + card_erro (máx. 500 caracteres)
- telas distintas para declined e unavailable
- carrinho continua preenchido para tentar novamente

Se o cliente clicar “Pagar via Pix agora”:

- mantém o pedido de cartão como cartao_recusado
- calcula o valor PIX com 10% de desconto
- POST /api/orders/create cria OUTRO pedido com metodo_pagamento pix
- copia nome, email, telefone, endereço completo, produtos e tracking
- remove orderId e dados de cartão antes de criar o novo registro
- salva o novo orderId no sessionStorage
- /api/pix/create vincula QR Code, transaction_id e gateway ao novo pedido
- admin mostra dois registros próximos: cartão recusado + PIX gerado/pago

Resultado esperado no admin: o registro original continua com método Cartão, status Cartão recusado, dados e motivo da tentativa. Acima dele aparece um novo registro PIX com os mesmos dados cadastrais, endereço, itens e atribuição, mas com valor PIX, horário próprio, transaction_id e gateway. A conciliação consulta somente o registro PIX e o altera para Pago quando o provedor confirmar.

7.3 Página de rastreio

Cliente digita código em /rastrear-pedido

- Busca rastreio_origem.origem_at
- Timeline simulada (postado → trânsito → hub → saiu p/ entrega)
- Endereço vem de leads (por codigo_rastreio)
- Taxa de reenvio: modal PIX R\$ 9,00 + upload comprovante

7.4 Painel Admin (/admin)

- Login server-side com `ADMIN_USER` / `ADMIN_PASS` ; token HMAC expira em 8 horas e fica no `sessionStorage`
- Lista de pedidos com busca, filtros por status/data, paginação (50/página)
- Filtro "Pagos sem rastreio"
- Resumo financeiro (recebido hoje / últimos 7 dias)
- Dropdown de status — marcar "pago" dispara fluxo completo (com confirmação)
- Coluna gateway PIX (IronPay, UmbrellaPag, etc.)
- Status "Reembolsado"
- Gerador de código de rastreio (recolhível) + WhatsApp + email
- Modal Gateways — toggle PIX/cartão
- Visualização de cartão criptografado (`VITE_ENCRYPT_KEY`)
- Recuperação de carrinho manual + automática (cron)
- Botão Sair (logout)
- Atualização automática a cada **60 segundos** e botão Atualizar manual
- A conciliação de PIX roda em paralelo; a lista antiga permanece visível durante refresh
- Somente a carga inicial mostra spinner grande; refresh usa apenas o ícone do botão
- Bloqueio contra chamadas simultâneas e timeout de 10 segundos na reconciliação disparada pelo admin

Algoritmo obrigatório da atualização do admin

```
Ao montar: fetchLeads(); setInterval(fetchLeads, 60_000)
Se já existe chamada em andamento: retornar sem duplicar
Primeira carga: mostrar loader; cargas seguintes: manter leads visíveis
Em paralelo: POST /api/reconcile-payments (Bearer token, abortar após 10s)
Buscar leads ordenados por created_at DESC
Se reconciliação concluir com sucesso: buscar novamente silenciosamente
Falha de refresh: preservar dados já carregados
```

7.5 Recuperação de carrinho

```
Cron (cron-job.org) a cada 15 min
→ POST /api/process-recovery-queue
→ considera somente PIX gerado entre 30 min e 24h atrás
→ Email 1: 30 min após PIX gerado
→ Email 2: +4 horas
→ Email 3: +24 horas após o email 2
→ para automaticamente quando status vira pago
```

- Processa no máximo 10 entradas novas e 10 follow-ups por execução.
- Deduplica por email: apenas o lead mais recente inicia sequência; duplicados recebem `recovery_count=-1` .

- Antes de enviar, faz update condicional no banco como trava. Duas execuções simultâneas não enviam o mesmo email.
- Se envio do email 1 falhar, devolve o lead para `recovery_count=0` . Se follow-up falhar, volta ao contador anterior e agenda nova tentativa em 30 minutos.
- Links dos emails usam `utm_source=recovery` , `utm_medium=email` e campanhas `email1` / `email2` / `email3` , alimentando a atribuição descrita em 7.6.

7.6 Atribuição UTM e clientes recuperados

- `saveTrackingParams()` salva `utm_source` , `utm_medium` , `utm_campaign` , `utm_content` , `utm_term` , `gclid` , `fbclid` e `ttclid` no `localStorage` .
- O checkout envia esses dados ao endpoint server-side junto com `ga_client_id` .
- O servidor procura pedidos do mesmo email ou telefone nos últimos 30 dias. Visita atual sem UTM herda a atribuição anterior e recebe `recovered_via_whatsapp: true` .
- UTMify recebe valores em centavos, país BR, produto, cliente, método, datas, comissão e parâmetros de tracking.
- Ao gerar PIX envia `waiting_payment` ; após confirmação envia `paid` com `approvedDate` .

7.7 Barra de frete e localização

- Exibe prazo visual de 30 minutos a partir da abertura e tenta personalizar com a cidade.
- Usa `navigator.geolocation.watchPosition` com alta precisão, sem cache e limite de 30 segundos.
- Aceita coordenada somente com precisão de até 500 metros e faz geocodificação reversa pelo Nominatim/OpenStreetMap.
- Se o usuário negar a permissão (ou o navegador não tiver geolocalização), usa `ipapi.co` como fallback. Se GPS autorizado for impreciso ou der outro erro, não substitui pela cidade do IP.
- Prioridade do nome: city → town → village → municipality → county.
- A barra não aparece em `/sucesso` nem em `/rastrear-pedido` .

8. O que personalizar na nova loja

8.1 Produtos — `src/lib/data.ts`

Edite o array `products[]`. Cada produto precisa de:

- ☐ id, slug (URL), name
- ☐ price, originalPrice, pixPrice (10% desconto PIX), installment
- ☐ rating, reviewCount, soldCount, badge (opcional)
- ☐ description, features[]
- ☐ mainImage, gallery[] (caminhos em `/public/images/`)
- ☐ productReviews[] (avaliações com author, rating, text, photo opcional)

8.2 Imagens — `public/images/`

- ☐ Fotos dos produtos (.webp recomendado)
- ☐ Fotos das avaliações (rev-*.webp)
- ☐ logo.webp → logo da loja
- ☐ hero.webp → imagem do banner principal
- ☐ favicon.svg

8.3 Textos e identidade

- ☐ `index.html` — título, meta description, GTM/GA4/UTMify IDs
- ☐ `src/components/layout/header.tsx` — logo, nome
- ☐ `src/components/layout/footer.tsx` — links, contato
- ☐ `src/pages/home.tsx` — hero, títulos, provas sociais
- ☐ `src/components/frete-gratis-bar.tsx` — texto, prazo e regras de localização
- ☐ Páginas institucionais (duvidas, contato, política)
- ☐ Emails em `functions/api/send-tracking-email.js` e `send-recovery-email.js`
- ☐ Prefixo do código de rastreio: manter consistente nos geradores do admin, checkout e backend `leads-store.js`
- ☐ `functions/_lib/utmify.js` — IDs/nome padrão do produto e domínio de email fallback

8.4 Credenciais e contas (criar novas para cada loja)

- ☐ Supabase — projeto novo
- ☐ Cloudflare Pages — projeto novo
- ☐ IronPay / MasterFy / UmbrellaPag / Venus Pay — credenciais da loja
- ☐ Resend — domínio verificado
- ☐ Facebook — pixel + token CAPI novos
- ☐ UTMify — token (pode reusar ou criar novo)
- ☐ Domínio próprio apontando para Cloudflare

9. Lógica que deve ser implementada exatamente

Não improvisar estes fluxos. Implementar como descrito neste guia:

- ☒ Toda a lógica de checkout (PIX + cartão)
- ☒ Cloudflare Functions (`functions/api/` e `functions/_lib/`)
- ☒ Painel admin completo
- ☒ Sistema de rastreo (leads + rastreo_origem)
- ☒ Emails automáticos (só muda branding/textos)
- ☒ Recuperação de carrinho + cron
- ☒ FB CAPI + UTMify (só mudam tokens)
- ☒ Criação server-side do pedido antes de chamar qualquer gateway
- ☒ Atribuição current/first/last touch com janela de 30 dias
- ☒ Conciliação PIX autenticada e fulfillment idempotente
- ☒ Atualização silenciosa do admin a cada 60 segundos
- ☒ Localização precisa com fallback por IP somente quando permitido pelas regras
- ☒ Componentes UI (shadcn) e animações
- ☒ Estrutura de rotas
- ☒ Rate limit PIX (KV)
- ☒ Gateway config (payment_gateways no Supabase)

10. Serviços externos e contas necessárias

Serviço	Gratuito?	Obrigatório?	Função
Repositório Git próprio	Sim	Sim	Código da nova loja + deploy automático no Cloudflare
Cloudflare Pages	Sim	Sim	Hospedagem + functions + KV
Supabase	Plano free	Sim	Banco de dados + storage
IronPay (ou outro PIX)	—	Sim (PIX)	Gateway PIX principal
Venus Pay	—	Sim (cartão)	Gateway cartão
Resend	Free tier	Sim	Emails transacionais
cron-job.org	Sim	Sim	Cron recuperação carrinho
Facebook Ads	—	Opcional	Pixel + CAPI
UTMify	—	Opcional	Rastreamento UTM's
Google Tag Manager	Sim	Opcional	Analytics
ViaCEP	Sim	Automático	Busca CEP no checkout
Nominatim / OpenStreetMap	Sim	Automático	Converte GPS preciso em cidade
ipapi.co	Sim/limitado	Fallback	Cidade aproximada quando GPS é negado ou indisponível
Binlist	Sim/limitado	Opcional	Bandeira/categoria exibida no visualizador do admin

11. Checklist de testes antes de ir ao ar

- ☐ Site carrega na home com produtos corretos
- ☐ Adicionar ao carrinho + ir ao checkout
- ☐ PIX: QR Code gera + status atualiza após pagamento
- ☐ Pedido: apenas um lead é criado e o mesmo `orderId` recebe `transaction_id/gateway`
- ☐ Fallback cartão → PIX: cartão recusado permanece e um segundo lead PIX é criado com os mesmos dados cadastrais/endereço/produtos/tracking
- ☐ Fallback cartão → PIX: o novo lead recebe horário, valor com desconto, `transaction_id` e `gateway` próprios
- ☐ Fallback cartão → PIX: clique duplo é bloqueado e falha ao criar o segundo pedido não gera cobrança órfã
- ☐ PIX pendente aparece na UTMify como `waiting_payment`
- ☐ PIX pago por polling, webhook e reconciliação não duplica rastreio, email nem conversão
- ☐ Webhook falso não marca pedido pago sem consulta válida ao provedor
- ☐ Cartão: aprovação/recusa funcionam
- ☐ Admin: login, listar pedidos, filtrar, marcar pago
- ☐ Admin: Gateways — toggle PIX e cartão
- ☐ Rastreio: código gerado aparece em `/rastrear-pedido`
- ☐ Email de rastreio chega (Resend)
- ☐ Webhook PIX configurado no gateway
- ☐ Cron de recuperação responde (POST com `CRON_SECRET`)
- ☐ Mobile: admin não corta botões no topo
- ☐ Admin: aguardar mais de 60s; lista continua visível e apenas ícone Atualizar gira
- ☐ Admin: uma reconciliação lenta termina/aborta em até 10s sem travar contatos
- ☐ Domínio customizado + `SITE_URL` correto
- ☐ FB Purchase e UTMify (se usar ads)
- ☐ UTM: pedido com parâmetros atuais grava `current_visit` + `first/last touch`
- ☐ UTM: mesmo cliente volta sem parâmetros em até 30 dias e recupera atribuição anterior
- ☐ Localização: GPS preciso mostra município; permissão negada usa IP; GPS impreciso não exibe cidade errada
- ☐ Executar `npm run build` e revisar console/rede nas rotas principais

12. Prompt pronto para usar com um assistente de IA

Copie e cole ao iniciar um novo projeto com Cursor, Replit ou similar:

Crie uma loja e-commerce NOVA e 100% funcional seguindo o PDF anexo (especificação técnica). Não copie nenhum site existente. Não procure repositórios, zips ou lojas de referência na internet.

NOVA LOJA:

- Produto: [ex: capacetes motocross]
- Nome: [ex: Capacetes Pro]
- Domínio: [ex: capacetespro.com.br]
- Prefixo rastreo: [ex: CP]

SIGA O GUIA À RISCA:

1. Criar o projeto do zero (Seção 2 e 3)
2. Supabase novo + SQL das tabelas (Seção 4)
3. Produtos, imagens e textos DESTA loja (Seção 8)
4. Cloudflare Pages + env vars + KV (Seção 5)
5. Gateways + webhooks (Seção 6)
6. Resend + cron-job.org (Seções 3 e 7)
7. Implementar exatamente a lógica das Seções 7, 9 e 14
8. Preservar criação server-side, atribuição UTM de 30 dias e idempotência
9. Fazer o checklist integral da Seção 11

Credenciais serão fornecidas separadamente. Não commitar secrets.

Antes de declarar pronto, entregue: árvore de arquivos, SQL, lista de env vars, URLs de webhook/cron, resultado do build e evidências dos testes críticos.

13. Problemas comuns e soluções

Problema	Causa provável	Solução
PIX não gera QR Code	Gateway off ou credencial faltando	Verificar Gateways no admin + env vars no Cloudflare
Webhook não atualiza pedido	SITE_URL errado ou webhook não configurado	Atualizar SITE_URL + URL webhook no gateway
PIX pago continua pendente	Polling fechou, webhook falhou ou gateway mudou	Executar reconciliação autenticada e conferir transaction_id + gateway salvo no lead
UTMify não atribui venda	Pedido criado sem tracking, token ausente ou purchase_sent inconsistente	Conferir tracking JSON, UTMIFY_API_TOKEN, logs e restaurar purchase_sent=false somente após falha confirmada
Pedido duplicado	Frontend ainda insere lead e também chama /api/orders/create	Remover insert paralelo; criar somente pelo endpoint e reutilizar orderId
Rastreo não encontra código	Falta registro em rastreo_origem	Salvar código nas duas tabelas ao marcar pago
Admin não loga	ADMIN_USER/PASS/SECRET faltando	Configurar 3 vars no Cloudflare + redeploy
Emails não chegam	Domínio não verificado no Resend	Verificar DNS SPF/DKIM no Resend
Build falha no Cloudflare	Node version ou dependências	Node 20 + npm install + npm run build local
Cartão desabilitado no checkout	Gateway venuspay off no admin	Ativar Venus Pay em Gateways
Rate limit PIX (429)	5 tentativas/IP/hora	Aguardar 1h ou testar de outro IP
Restore Supabase com erro	Projeto não estava vazio	Restaurar só em projeto novo sem SQL prévio
VITE_* não funciona	Vars adicionadas após build	Redeploy após adicionar vars VITE_*
Admin apaga lista a cada minuto	Mesmo estado loading usado na primeira carga e no refresh	Separar loading/refreshing, manter leads e bloquear concorrência
Admin fica girando indefinidamente	Reconciliação bloqueando busca	Rodar em paralelo e abortar após 10s

Cidade errada na barra	IP sobrescreveu GPS ou coordenada estava imprecisa	Aceitar GPS até 500m; não usar IP depois de coordenada autorizada imprecisa
------------------------	--	---

14. Contratos de dados, estados e regras de idempotência

Momento	Estado/efeito obrigatório	Proteção
Checkout validado	<code>orders/create</code> cria lead <code>checkout_iniciado</code>	Sem orderId, parar o fluxo e avisar usuário
Cartão recusado → PIX	Preservar o cartão e criar outro lead PIX com os mesmos dados	Novo orderId antes do QR Code; remover dados do cartão do novo pedido; bloquear clique duplo
PIX criado	Atualizar o mesmo lead para <code>pix_gerado</code> , <code>transaction_id</code> e gateway	orderId obrigatório; máximo 5/IP/h
PIX consultado	Normalizar paid/approved/pago, expired/cancelled/canceled/refunded ou pending	Usar gateway salvo no pedido, não apenas o gateway atualmente ativo
PIX confirmado	<code>pago</code> + rastreio + origem + email + UTMify paid	<code>codigo_rastreio IS NULL</code> e <code>purchase_sent=false</code>
Webhook recebido	Identificar provedor/transação e responder 200	Consultar API privada antes de alterar status
Admin aberto	Atualizar leads e conciliar pendentes a cada 60s	Token HMAC, chamada única, lista preservada, timeout 10s

Chaves de armazenamento do navegador

Chave	Tipo	Uso
<code>loja-figurinhas-cart</code>	localStorage	Carrinho persistente
<code>topmix_buyer</code> / <code>topmix_address_loja</code>	localStorage	Repreencher comprador e endereço
8 parâmetros de atribuição	localStorage	UTM, gclid, fbclid e ttclid
<code>topmix_pix_tentativas</code> / <code>topmix_card_tentativas</code>	localStorage	Contadores visuais por cliente/aparelho
<code>checkoutData</code> , <code>pixResult</code> , <code>cardResult</code> , <code>cardData</code>	sessionStorage	Transição checkout → sucesso
<code>adm_token</code>	sessionStorage	Sessão administrativa assinada

15. Controle de versão e critério de conclusão

Fonte de verdade: este guia descreve o comportamento validado em 04/09/2026, incluindo a correção local mais recente do refresh do admin. Se o código mudar, atualizar primeiro o inventário de rotas, variáveis, tabelas, estados, integrações e checklist; depois regenerar o PDF.

Uma recriação só está concluída quando:

1. O build de produção termina sem erro.
2. Todas as rotas frontend e APIs da Seção 1 existem.
3. Schema, Storage, RLS, variáveis e KV estão configurados.
4. Os quatro provedores PIX e o cartão respeitam toggles e contratos.
5. Polling, webhook e reconciliação convergem para o mesmo resultado sem duplicidade.
6. A troca de cartão recusado para PIX produz dois registros separados e a conciliação atua no registro PIX.
7. Admin, rastreo, emails, recuperação, UTMify, CAPI, localização e responsividade passam no checklist.
8. Nenhum segredo aparece no Git ou no bundle do frontend.

Documento revisado em 04/09/2026 — Especificação técnica e operacional da TopMix com 5 configurações de gateway, persistência server-side, conciliação PIX, atribuição UTM de 30 dias, admin atualizado, rastreo, recuperação e deploy Cloudflare Pages.

Para recriar em outro nicho, preserve os contratos funcionais e substitua apenas marca, catálogo, ativos, textos, contas e credenciais. Nunca copie conteúdo de terceiros sem autorização.